Paper Trail: Design Document

Course: CS 5610 Web Development, Northeastern University, Fall 2026
Team: Mayank Devangkumar Tamakuwala, Varshith Sai Pothula
Repository: https://github.com/MayankTamakuwala/paper-trail
Live app: https://paper-trail-mu.vercel.app

---

1. Project Description

The problem

Graduate students do not have a problem finding research papers. Search is solved. Google Scholar, arXiv, Semantic Scholar and a dozen other tools will hand you more relevant work than you could read in a decade.

The problem is what happens after you find them.

A typical second year master's student has somewhere between forty and a hundred papers they have decided are relevant to their thesis. Those papers live in four or five places at once: browser tabs that have been open so long the favicon is the only way to tell them apart, a downloads folder full of files named `2310.08419v2.pdf`, a Zotero library opened twice and abandoned, a Slack message to themselves, and a note in their phone. None of these places record the one thing that actually matters, which is **why** the paper seemed important at the moment they saved it.

Three weeks later the paper is still there and the reason is gone. So the student either re-reads the abstract to reconstruct their own thinking, or more commonly, skips it. The backlog grows, it becomes a source of guilt rather than a resource, and eventually they stop looking at it entirely.

The solution

Paper Trail is a reading queue, not a search engine. It assumes you have already found the papers and helps you finish them.

It does four things:

1. **Captures intent at save time.** Every entry carries a priority and a free text note answering "why are you saving this." That note is the single highest value field in the system, because it is the thing every other tool throws away.
2. **Makes the backlog visible.** Papers move through three states, to read, reading, and read, with counts always on screen. Seeing "47 to read, 2 reading, 6 read" is

uncomfortable, and that discomfort is the point. You cannot triage a pile you cannot measure.
3. **Supports a shared catalog.** A searchable catalog of over 1,000 real papers, seeded from OpenAlex, so finding something to add does not require leaving the app. The catalog is collaboratively maintained, so wrong metadata can be corrected rather than worked around.
4. **Removes the retyping.** Any paper in the catalog can be pushed into your reading list with one button, so the common path costs a single click rather than a filled form.

Scope for Project 2

Two MongoDB collections with full CRUD, a vanilla JavaScript client side rendered frontend, and an Express JSON API, deployed as a single Vercel project.

In scope	Out of scope for now
Personal reading list with full CRUD	User accounts and passwords
Shared paper catalog with search and CRUD	PDF storage or full text
Per browser identity so lists stay private	Social features and sharing
Over 1,000 seeded papers	Citation graph and recommendations

Why this is useful

The test for this kind of project is whether the author would use it. Both of us are graduate students with exactly this problem. The reading list is the feature we would open on a Monday morning to decide what to work on, and the note field is the reason we would keep coming back rather than drifting to a spreadsheet.

---

2. User Personas

Persona 1: Mira Chandrasekaran, the thesis writer

> "I have sixty papers saved and I have read maybe twelve. I do not know which twelve matter."

Age and role	24, second year master's student in computer science
Context	Writing a thesis on efficient transformer inference. Four months from her defense
Technical comfort	High. Writes code daily, but has no patience for configuring a tool
Devices	Laptop almost exclusively, occasionally her phone on the train

Goals

- Know which saved papers are load bearing for her argument and which are optional
- Remember why she saved something without re-reading the abstract
- Walk into an advisor meeting able to say what she has read this week

Frustrations

- Her reference manager shows her everything at once, with no sense of priority
- She has saved the same paper three times from three different places
- When she opens a saved PDF she often cannot recall what she was looking for in it

How Paper Trail fits into her week

Mira opens the app on Monday morning before her advisor meeting, sorts her list by priority, and reads the top five notes. That takes her about ninety seconds and tells her what to work on, which is the whole point. The feature she actually depends on is the note field, because it is the only place in her entire workflow where her reasoning survives contact with time. Every other tool she has tried stores the paper and discards the thought. Priority and status matter to her too, but mostly as a way of sorting the notes. If she had to give up everything except one field, she would keep the note and rebuild the rest around it.

Persona 2: Daniel Osei, the first year PhD student

> "Every paper I read cites four more. I need to know when to stop."

Age and role	26, first year PhD student, three weeks into a literature review
Context	Has no thesis topic yet. Reading broadly to find one
Technical comfort	High, but new to the research workflow
Devices	Laptop and a tablet for reading

Goals

- See the shape of his backlog so he can tell whether it is growing or shrinking
- Distinguish "read this to understand the field" from "read this if I pick this topic"
- Feel like progress is being made during a phase that feels bottomless

Frustrations

- The literature review has no obvious end condition
- He cannot tell whether he is being thorough or just procrastinating
- Flat lists give no feedback, so the work feels invisible

How Paper Trail fits into his week

Daniel uses the app differently from Mira. He is not prioritizing, he is surveying, and the thing he needs is a sense of scale. The status counts sitting above his list give him that. Watching "to read" grow in week one tells him the field is bigger than he thought; watching it flatten in week four tells him he is approaching the edges of it. That single line of text is the only feedback a literature review gives you, and no other tool he has used puts it on screen. He also leans on tags more than the other personas, because he is still figuring out which clusters of work exist, and tagging as he goes is how those clusters become visible.

Persona 3: Dr. Adaeze Okafor, the course instructor

> "I rebuild this reading list every spring and I never remember why I cut things."

Age and role	41, associate professor, teaches a graduate seminar
Context	Assembles a twelve week reading list each term
Technical comfort	Moderate. Uses tools that work and abandons ones that do not
Devices	Laptop, often with twenty tabs open

Goals

- Search a broad catalog by field and year to find seminar candidates
- Record why each paper earned its slot, so next year is editing rather than restarting
- Keep a stable list she can return to months later

Frustrations

- Last year's reasoning exists only in her memory, which has moved on
- Search tools optimize for novelty, but she needs foundational work too
- Every tool wants her to adopt a whole workflow for one annual task

How Paper Trail fits into her year

Dr. Okafor is a seasonal user. She opens the app hard for two weeks in December, then does not touch it again until the following December. That shapes what she needs: the catalog search has to be good enough to work without relearning, and her notes have to still make sense a full year later. The year range filter matters more to her than to anyone else, because she is deliberately looking for work that has had time to settle rather than the newest preprints. The "add to reading list" button on each result is what makes her two week sprint tolerable, since she is moving dozens of papers and typing each one would end the exercise early. When she returns next December, the list she built this year is her starting point, and the notes explaining why each paper earned its slot mean she is editing a reading list rather than rebuilding one from nothing.

Persona 4: Sam Reyes, the research assistant

> "The year is wrong on half of these and nobody can fix it."

Age and role	23, research assistant in a systems lab
Context	Maintains the lab's shared reading material for six people
Technical comfort	High. Notices data quality problems others ignore
Devices	Laptop

Goals

- Add papers that automated indexes do not have, such as preprints and internal reports
- Correct wrong metadata rather than working around it
- Remove duplicate entries so searches return clean results

Frustrations

- Automated catalogs carry wrong years, missing authors, and duplicate records
- Most tools treat the catalog as read only, so errors are permanent
- Internal technical reports exist in no index at all

How Paper Trail fits into his week

Sam is the only persona who cares about the catalog as a shared artifact rather than as a search box. He treats it the way a maintainer treats a wiki: he notices when something is wrong and he expects to be able to fix it. Giving him edit and delete rights on the papers collection is what makes the catalog improve over time instead of slowly rotting. Without him the data only ever gets worse, because automated indexes do not correct themselves and every user who hits a bad record just works around it. His work is invisible to the other three personas, which is exactly how he would describe a catalog that is doing its job.

---

3. User Stories

Stories are split so the two of us work on independent collections. Neither set depends on the other being finished.

Mayank Tamakuwala, the `readingList` collection

Story 1: Saving a paper with its reason

It is Tuesday night and Mira is deep in a related work section when she finds a paper on quantization that looks like it might replace an argument she was going to make herself. She does not have time to read it now. She opens Paper Trail, types the title and authors, sets priority to **high**, and in the note field writes "might make section 3.2 unnecessary, check before rewriting." Three weeks later, when she opens her list to plan her week, that sentence is still there and she knows instantly why this one goes first.

Acceptance criteria

- A form accepts title (required), authors, year, venue, status, priority, tags and a note
- Submitting with an empty title shows an inline error and does not create an entry
- A created entry appears in the list without a page reload
- Priority is visible on the entry at a glance
- An empty list offers to load a few sample papers, so a new user can see the behavior before typing anything

Story 2: Moving papers through the backlog

Daniel opens his list on Monday. The counts read "31 to read, 1 reading, 8 read." He picks the paper he started on Friday, clicks **Mark as Read**, and watches the counts become 31, 0 and 9. Then he filters to **To read** to pick the next one, and the other thirty nine entries get out of his way. By Friday the "read" number has moved to 12 and, for the first time since he started, the literature review feels like it has a direction.

Acceptance criteria

- Each entry has a single button advancing it to read, then reading, then read, then back
- Status counts appear on screen and update after every change
- Filter buttons narrow the list to one status, with the active filter visibly marked
- The list can be sorted by date added, priority or title

Story 3: Keeping the list honest

Mira's argument has shifted. Two papers she marked high priority in September are no longer relevant, and one she almost deleted has become central. She clicks **Edit** on that one, changes its priority to high, and rewrites the note to reflect what she now needs from it. Then she deletes the two that no longer apply, confirming each so she does not lose one by mistake. Her list now reflects the thesis she is actually writing rather than the one she planned three months ago.

Acceptance criteria

- Edit loads the entry into the same form, which switches to an update mode
- Saving updates the existing entry rather than creating a duplicate
- Edit mode can be cancelled, restoring the form to its add state
- Delete asks for confirmation before removing an entry
- Entries are scoped per user, so one person's list is never visible to another

---

Varshith Sai Pothula, the `papers` collection

Story 4: Finding seminar candidates

Dr. Okafor is building next term's reading list. She searches the catalog for "distributed consensus", filters to the Computer Science field, and sets the year range to 2015 through 2020 because she wants work that has had time to be absorbed rather than this month's preprints. The results come back sorted by citation count, so the foundational papers surface first. She works through two pages, and for each paper worth keeping she clicks **Add to reading list** rather than retyping it.

Acceptance criteria

- A search field matches against title, authors and venue
- Results can be filtered by field and by a year range
- Results are paginated, with the total result count shown
- Each result shows title, authors, year, venue, citation count and abstract
- Each result has a button that copies the paper into the reading list, preserving the full author list rather than a shortened display form

Story 5: Adding what the index is missing

Sam's lab has a technical report that was never published anywhere, so no index has it. He opens the catalog, clicks **Add a paper**, and a dialog opens over the results with a form for the title, the three internal authors and the year. It is now searchable by everyone in the lab, sitting alongside the published work rather than in a separate folder nobody remembers.

Acceptance criteria

- A form creates a paper with title (required), authors, year, venue, DOI and abstract
- The form opens in a dialog rather than pushing the results down the page
- Papers added by hand are distinguishable from seeded ones
- A newly added paper appears in search results immediately

Story 6: Fixing the catalog

While searching, Sam notices a paper listed as 2019 that he knows was published in 2017, and a second entry for the same paper with a slightly different title. He edits the first to correct the year and deletes the duplicate. The next person who searches for it finds one clean, correct record.

Acceptance criteria

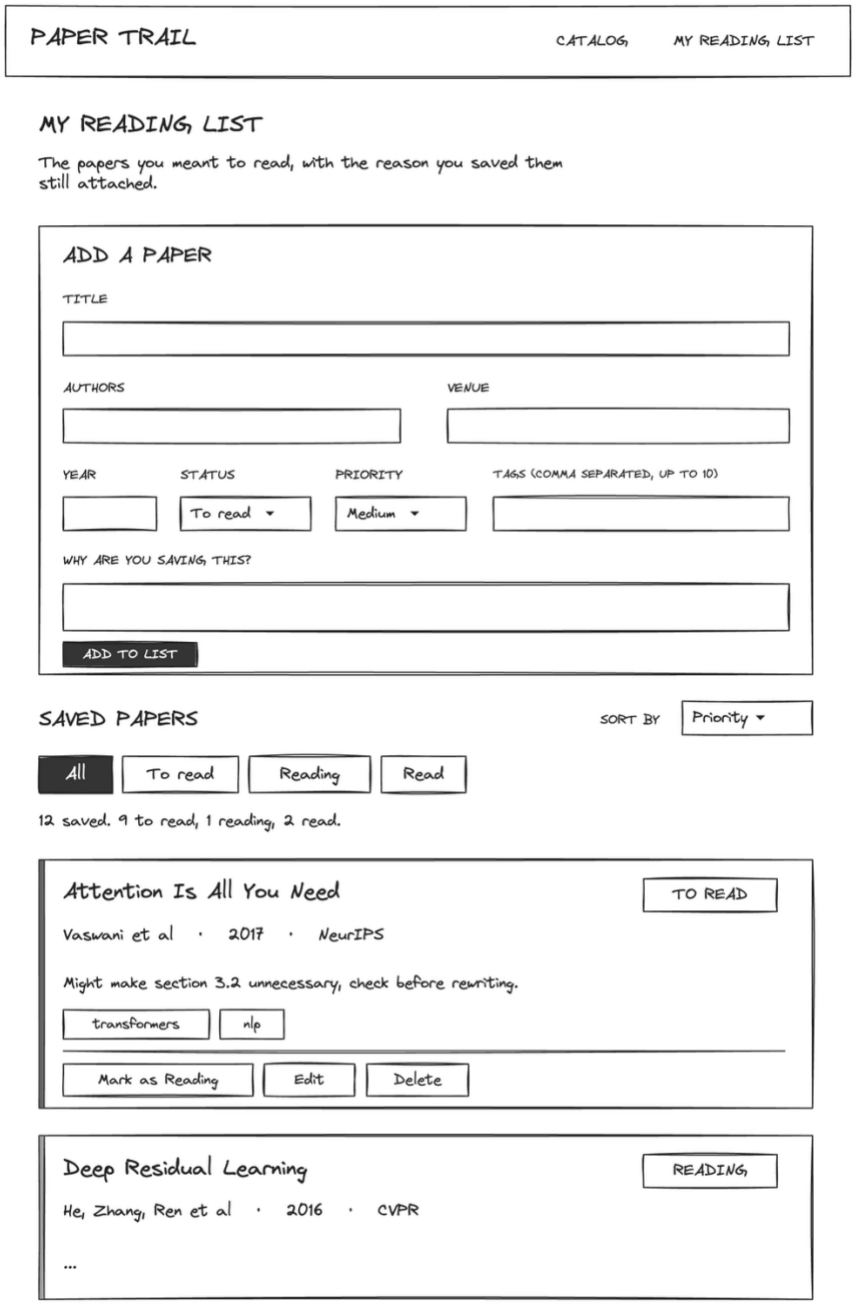
- Any catalog field can be edited and saved
- Deleting a paper asks for confirmation
- Changes are reflected in search results immediately

---

4. Design Mockups

Wireframes are low fidelity on purpose. At this stage the questions worth answering are about layout, hierarchy and which controls exist, not about pixels. A sketched style keeps reviewers commenting on structure rather than on border radius.

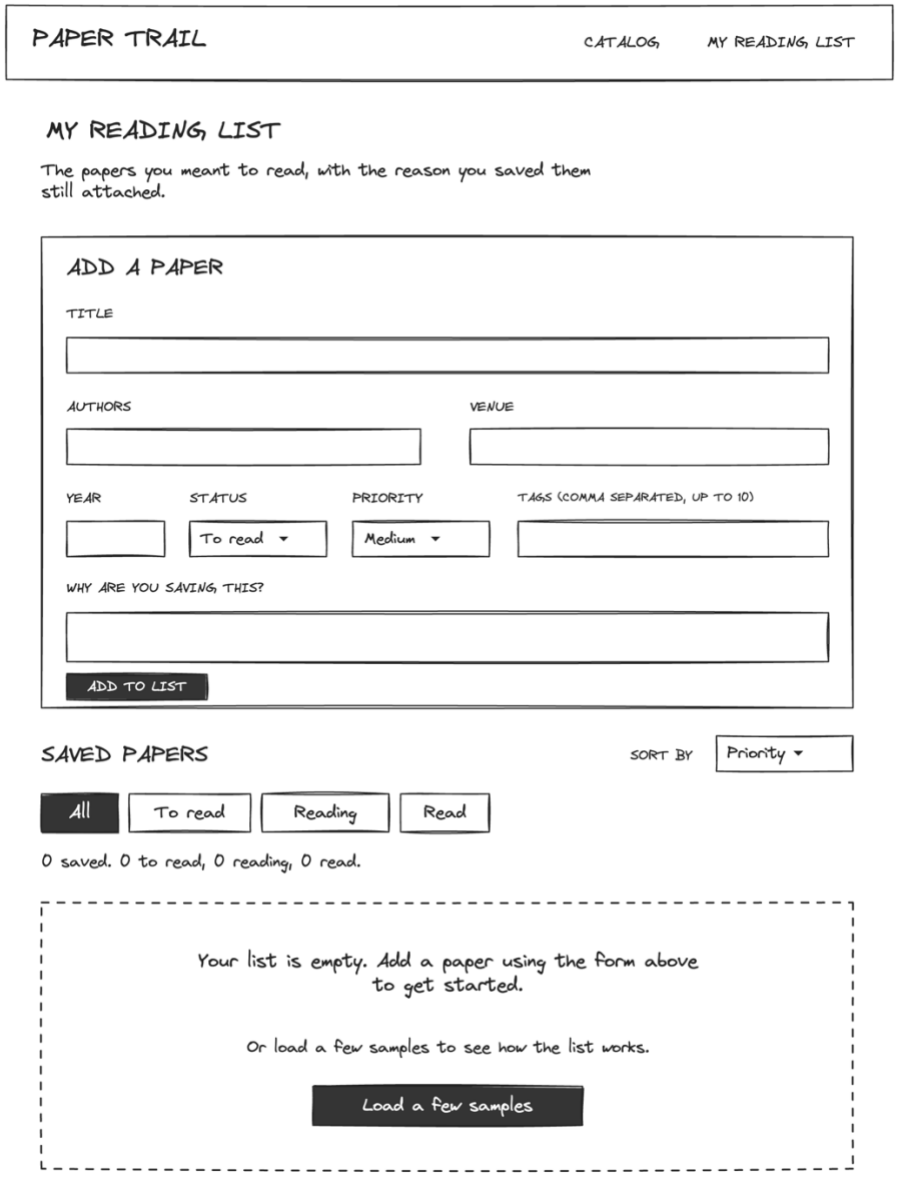
4.1 Reading list, populated state



The page is split into two stacked regions: a form that is always available at the top, and the list below it. The form does not hide behind a button or a modal, because adding a paper is the single most frequent action and anything that costs a click costs entries that never get saved.

Inside each entry, four things are visible without interaction: the title, the status pill, the note, and the tags. The note is given a full line of its own rather than being truncated to a caption, because it is the field the whole product exists to preserve.

4.2 Reading list, empty state

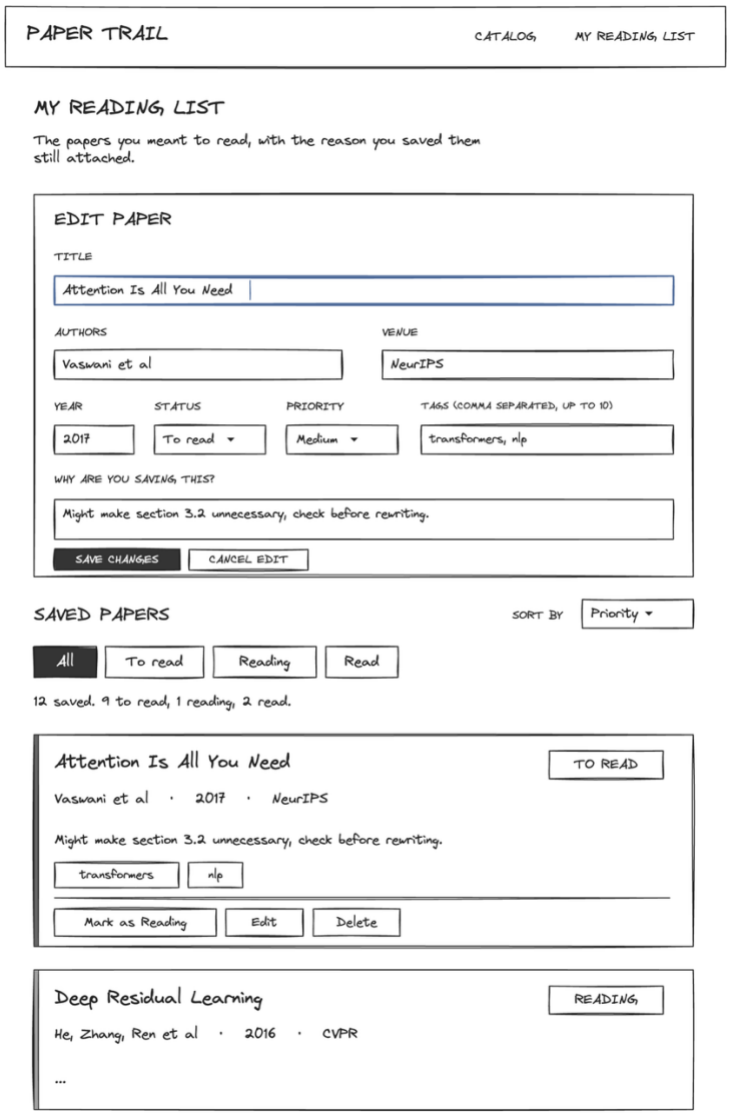


Every new user sees this first, so it teaches rather than apologizes. The dashed border signals a placeholder rather than a broken region, and the copy points back up at the

form instead of just announcing that nothing is here. It also offers a way in, for anyone who would rather see the behavior before typing anything.

When a status filter is active but matches nothing, the message changes to "Nothing in this status yet", so the user understands the list is filtered rather than empty. The sample button is not shown there, since seeding into a filtered view would be confusing.

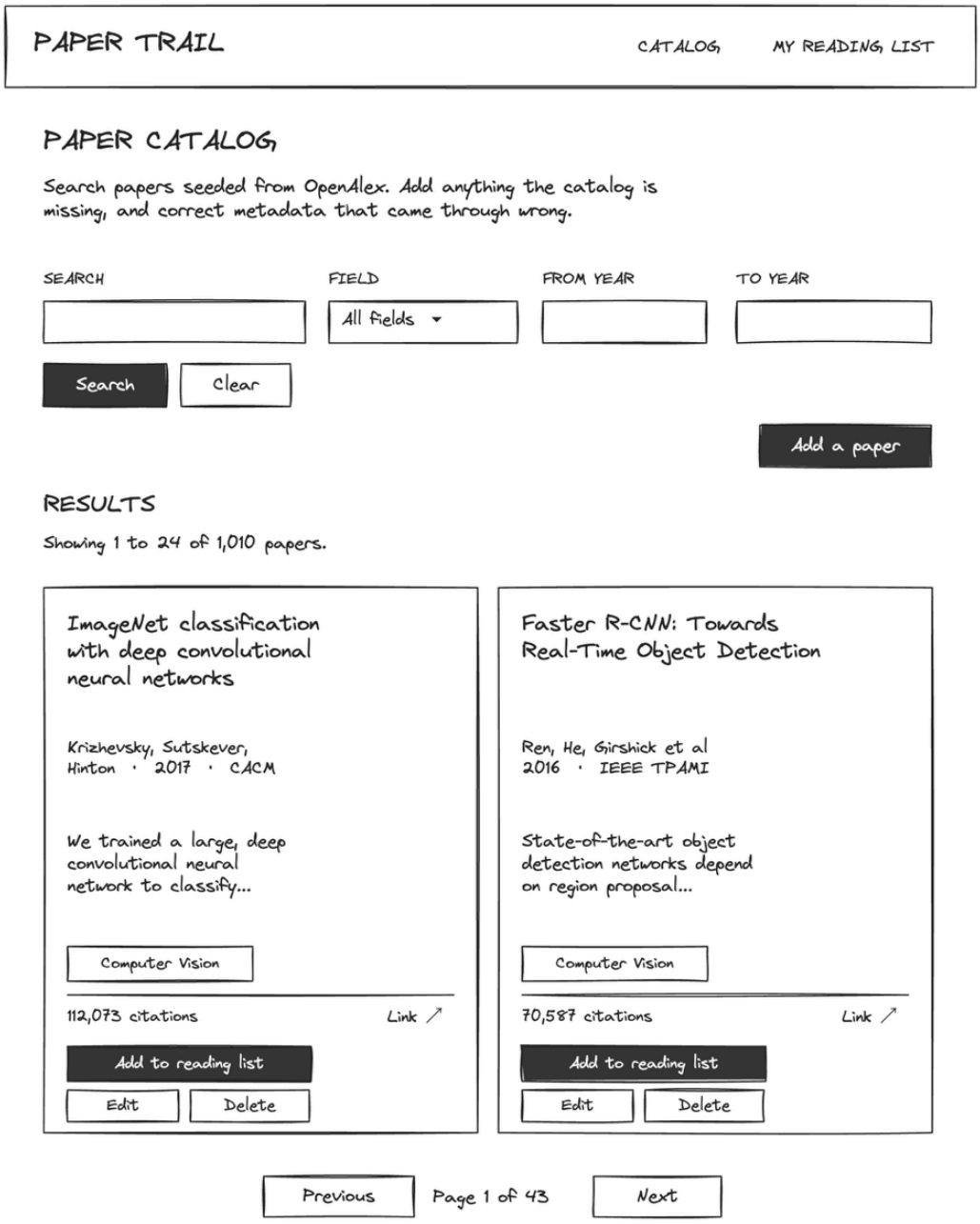
4.3 Reading list, edit mode



Editing reuses the add form rather than opening a modal. One form, two modes. Three things change: the heading becomes "Edit paper", the submit button becomes "Save changes", and a "Cancel edit" button appears beside it.

Reusing the form avoids building a focus trap and a restore point, and it means a user who has learned the add flow already knows the edit flow.

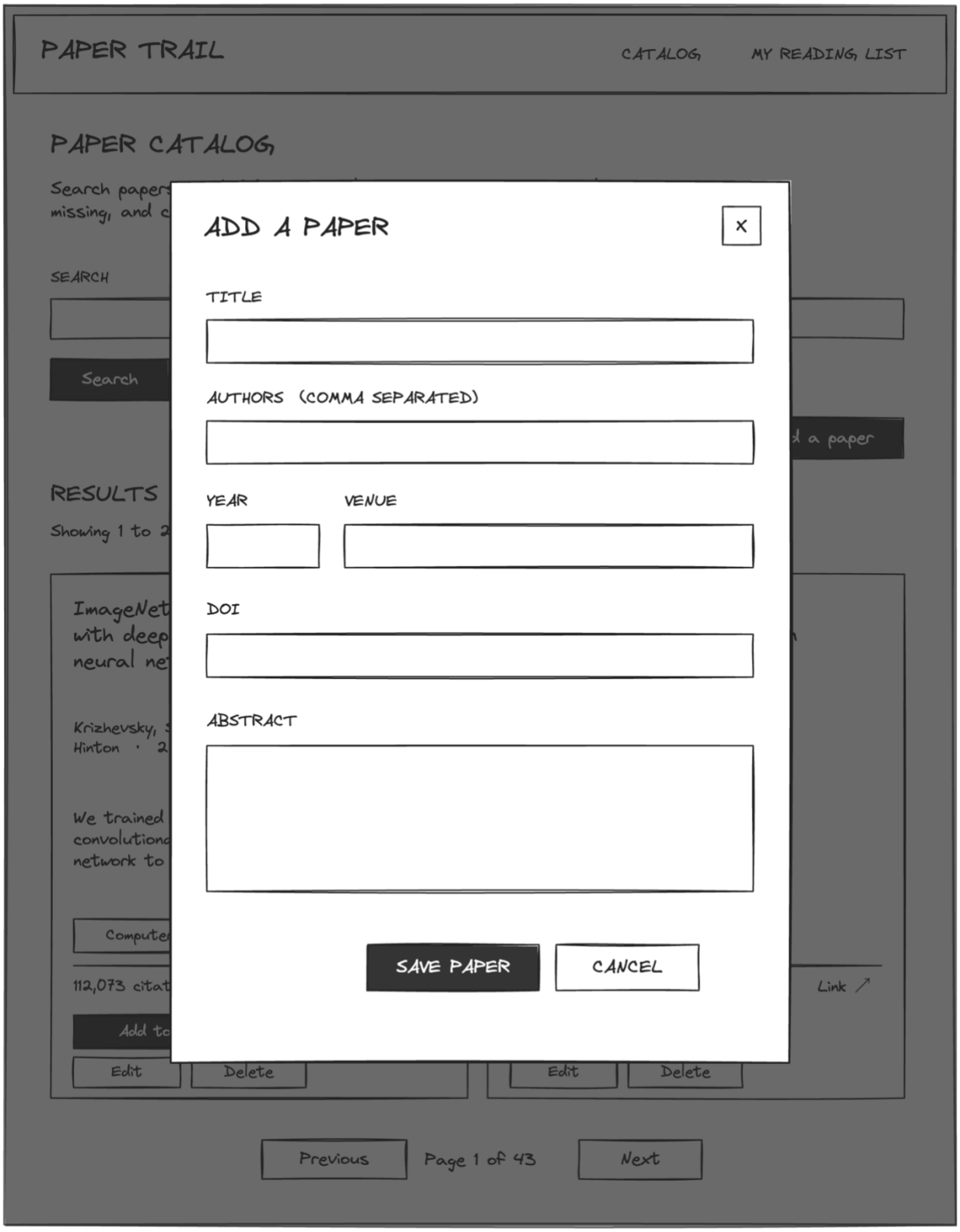
4.4 Catalog page



The catalog leads with its filters because arriving here means you are looking for something specific. Results are cards in a responsive grid rather than rows, since an abstract needs more horizontal room than a table gives it.

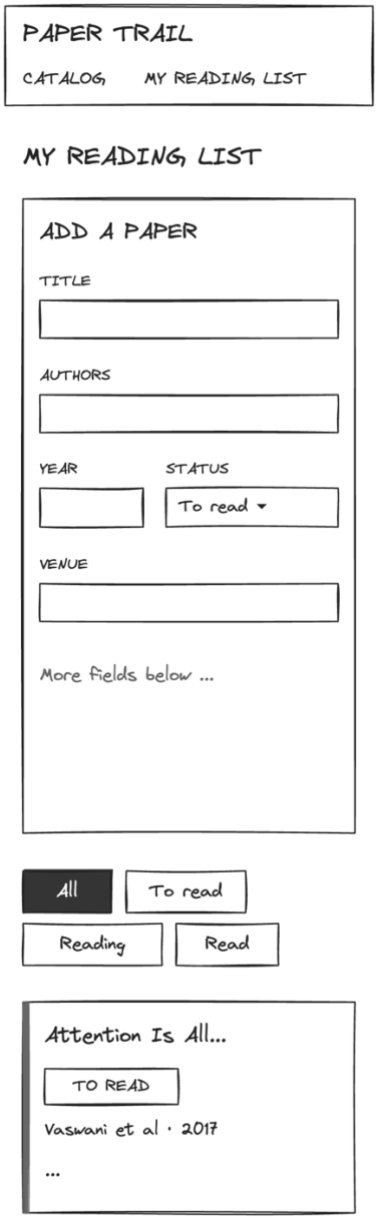
Results are sorted by citation count by default, which surfaces foundational work first. That matters for Dr. Okafor, who is looking for papers that have had time to settle rather than this month's preprints.

4.5 Catalog, add paper dialog



Adding a paper opens a native dialog over the results rather than pushing them down the page. The browser handles focus trapping, Escape to close, and making the background inert.

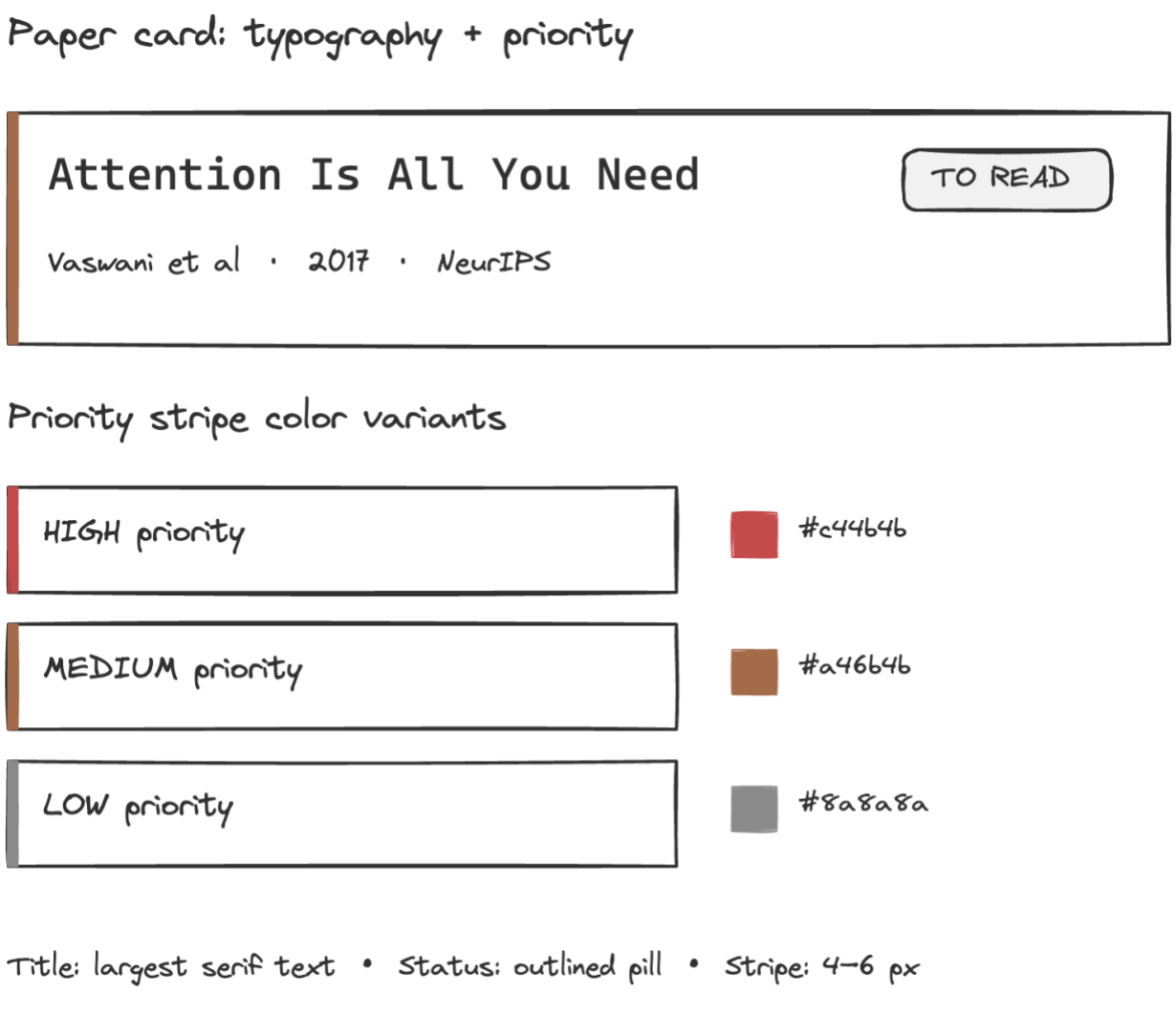
4.6 Mobile layout



Below 720px the form grid collapses from four columns to two, and below 640px the header navigation wraps beneath the brand and cards become a single column. The status filter chips wrap onto two rows rather than scrolling horizontally, so none of them are hidden.

There is no separate mobile page and no separate stylesheet, only two breakpoints inside the existing CSS modules.

4.7 Entry anatomy



The entry carries three levels of prominence. The title is the largest text and the only serif element, because it is the content. The status pill is outlined and small, because it is read once per entry. The priority stripe is 4 to 6px on the left edge.

Priority is encoded as a stripe rather than a word because priority is something you scan for down the whole list, while status is something you read on one entry at a time. Neither relies on color alone: the stripe is backed by the priority sort order, and the pill carries its own text.

---

5. Data Model

Two collections. The reading list stores its own copy of the paper details it needs, so the two are fully independent and neither page breaks if the other collection is empty.

papers

```
{
  _id:           ObjectId,
  openAlexId:    "https://openalex.org/W2741809807",
  title:         "Attention Is All You Need",
  authors:       ["Ashish Vaswani", "Noam Shazeer", ...],
  year:          2017,
  venue:         "Neural Information Processing Systems",
  doi:           "https://doi.org/10.5555/3295222",
  citationCount: 106322,
  fields:        ["Computer Science", "Artificial Intelligence"],
  abstract:      "The dominant sequence transduction models...",
  source:        "openalex" | "user",
  createdAt:     Date,
  updatedAt:     Date
}
```

readingList

```
{
  _id:          ObjectId,
  userId:       "a3f1c8e2-...",      <- anonymous browser id, generated and
  title:        "Attention Is...",      stored client side
  authors:      ["Ashish Vaswani", "Noam Shazeer"],
  year:         2017,
  venue:        "NeurIPS",
  status:       "to-read" | "reading" | "read",
  priority:     "high" | "medium" | "low",
  priorityRank: 0 | 1 | 2,           <- numeric copy so Mongo can sort
  tags:         ["transformers", "nlp"],
  note:         "Might make section 3.2 unnecessary...",
  createdAt:    Date,
  updatedAt:    Date
}
```

`authors` is a list in both collections so a paper can be pushed from the catalog into a reading list without losing anyone. Storing it as a display string would mean a paper with four or more authors arrives truncated.

Every entry carries a `userId`, so one person's reading list is never visible to another even though there are no accounts. The id is generated in the browser on first visit and kept in `localStorage`. Without it, a single deployed URL would mean one global list that every visitor could edit.

API surface

Method	Route	Purpose
GET	`/api/papers`	Search with `q`, `field`, `yearFrom`, `yearTo`, `page`, `limit`
GET	`/api/papers/meta/fields`	Field names and counts for the filter
POST	`/api/papers`	Add a paper
PUT	`/api/papers/:id`	Correct metadata
DELETE	`/api/papers/:id`	Remove a duplicate
GET	`/api/reading-list`	The caller's entries, filtered and sorted
POST	`/api/reading-list`	Save a paper
PUT	`/api/reading-list/:id`	Update status, priority, tags or note
DELETE	`/api/reading-list/:id`	Remove an entry

Reading list routes require an `x-user-id` header.

Input rules

Both collections share one validation helper, so authors, tags and fields behave consistently.

Rule	Behaviour
Whitespace around a value	trimmed silently
Empty values from a trailing comma	dropped silently
Duplicates, ignoring case	collapsed silently, first spelling wins

Rule	Behaviour
Brackets, braces, quotes, angle brackets	rejected with a message
A value longer than the field limit	rejected with a message
More entries than the field allows	rejected with a message

The split is deliberate. Typing a trailing comma or the same name twice is a slip the app should absorb quietly. Typing `["a","b"]` into a name field is a misunderstanding worth correcting out loud.

---

6. Visual Design

Typography

Role	Face	Rationale
Headings and paper titles	Georgia, serif	Paper titles are the content. A serif face signals "this is something to read" rather than "this is UI chrome"
Interface text, labels, buttons	system-ui, sans-serif	Controls should recede. The system face feels native and stays visually separate from content

The split is deliberate and load bearing. On a page that is mostly a list of titles, using one face for everything makes the titles compete with the buttons. Two faces create a clear hierarchy: serif is what you read, sans is what you click.

Color

Token	Value	Use
`--paper`	`#f7f6f3`	Page background, warm off white rather than pure

Token	Value	Use
		white
`--panel`	`#ffffff`	Cards and the form, lifted off the background
`--ink`	`#1b1d21`	Primary text
`--ink-soft`	`#5d6570`	Secondary text and labels
`--accent`	`#7a5c2e`	Primary actions, links, active states
`--danger`	`#a3332b`	Destructive actions and errors only
`--ok`	`#2f6b4a`	Read status
`--priority-high`	`#c44646`	High priority stripe
`--priority-medium`	`#a46646`	Medium priority stripe
`--priority-low`	`#8a8a8a`	Low priority stripe

The palette is warm and low contrast by design. This is a tool for an activity that already causes guilt, and a bright saturated interface would add urgency where none is wanted. The accent is a muted brown rather than the default blue, which also keeps links distinguishable from the status pills.

Approve and cancel are consistent everywhere: the primary action is a filled accent button, destructive actions are outlined in `--danger`, and secondary actions are plain outlined buttons.

Priority has its own three tokens rather than reusing `--danger` and `--accent`. The distinction matters: `--danger` is an action semantic meaning "this will destroy something", while a priority stripe is a data encoding meaning "this one matters more". Reusing one red for both would teach the user that red means two unrelated things, and a high priority card would read as a warning. Keeping them separate also means the

delete button and the high priority stripe are visibly different reds rather than nearly identical ones.

All three stripe colors clear the 3:1 non-text contrast ratio against the white card required by WCAG 1.4.11, at 4.87:1, 4.60:1 and 3.45:1 respectively. The low priority grey was darkened from a first draft at `#969696`, which measured 2.96:1 and would have failed.

---

7. Accessibility Decisions

Made up front rather than retrofitted. Retrofitting accessibility means rewriting markup, and the decisions below cost nothing if taken at the start.

- **Standard elements only.** Every control is a real `<button>`, `<input>`, `<select>` or `<textarea>`. Nothing is a styled `<div>`, so keyboard and screen reader behavior comes free.
- **The add paper dialog is a native `<dialog>`**, so focus trapping, Escape to close and an inert background are handled by the browser rather than by hand written JavaScript.
- **Every input has a `<label for>`**, not a placeholder standing in for one.
- **Editing a reading list entry happens inline, not in a dialog**, which avoids needing a focus trap and a restore point on the page where editing is most frequent.
- **Status messages use `role="status"`**, so changes to the counts are announced rather than appearing silently.
- **Form errors use `role="alert"`**, so a rejected submission is announced immediately.
- **Filters use `aria-pressed`** rather than a class alone, so their state is exposed to assistive technology.
- **A skip link** is the first focusable element on each page.
- **Visible focus rings** via `:focus-visible` on every interactive element.
- **Information is never color only.** Priority is a stripe backed by sort order, status is a pill with text.